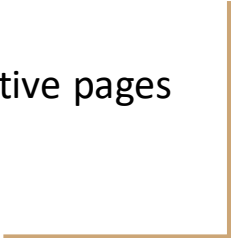




Web Application Development II

Week 4: Vue.js
Building reactive, interactive pages



Agenda

DOM Manipulation

Introduction to Vue

Installation and setup

Vue.js Features

- Data/Attribute/Form Input binding

- Class & Style binding

- Routing



What is DOM?

The browser represents an HTML document as Document Object Model (DOM).

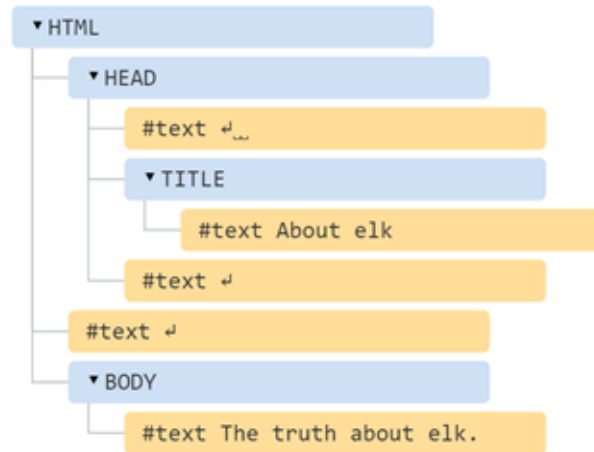
In DOM, every HTML entities – elements, attributes, texts are represented as nodes (objects) in a hierarchical data structure.

This data structure makes it easy for JavaScript to access and change the entities in an HTML document.

[Document Object Model \(DOM\) - Web APIs | MDN](#)

With DOM, we can access and modify the HTML document through Javascript.

```
<!DOCTYPE HTML>
<html>
  <head>
    <title>About elk</title>
  </head>
  <body>
    The truth about elk.
  </body>
</html>
```



DOM Manipulation

Manipulating the DOM, we can create dynamic web pages

- change all the HTML elements, attributes, and CSS styles in the page
- change all the HTML elements, attributes, and CSS styles in the page
- remove existing HTML elements and attributes
- add new HTML elements and attributes
- react to HTML events in the page
- create new HTML events in the page

You can do whatever you want to the tree.

What is a Reactive Application?

A reactive web application

- Observe changes in application state
- Propagate change notification throughout the application
- Render views automatically in response to changes in state
- Provide timely feedback for user interactions

Responsive

Resilient

Scalable

Message-driven

What is Vue.js

Vue (pronounced like View) is a progressive framework for building user interfaces.

The core library is focused on the view layer only, and is easy to pick up and integrate with other libraries or existing projects.

On the other hand, Vue is also perfectly capable of powering sophisticated Single-Page Applications when used in combination with modern tooling and supporting libraries.

Example

[Themes](#)

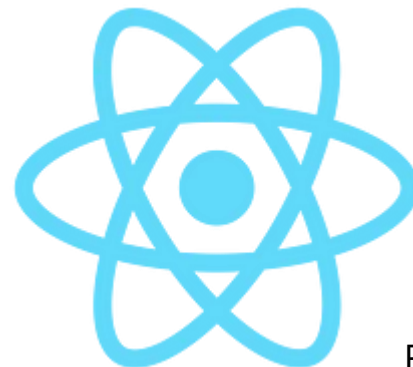
Why Vue.js



Used by
Alibaba



Google



Facebook

Why Vue.js

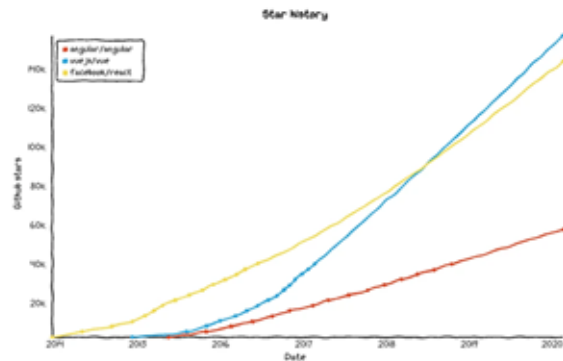
Extremely small (16KB)

Great runtime performance

Easy to learn

Popularity: [Read Here](#)

“React has been a leader in popularity among JavaScript frameworks. It takes the first place with the number of 48,718 dependents, whereas, Vue.js is the second most popular JavaScript framework with half as many dependents — 21,575, as reported by Node Package Manager. Though, if we take a look at statistics at the end of 2018, we will see that Vue’s dependents had tripled, while React’s had doubled.”



Vue.js Features

Data Binding: This feature helps manipulate or assign values to HTML attributes, change the style, assign classes with the help of binding directive called v-bind

Directives: built-in directives such as v-if, v-else, v-on, v-bind, and v-model, which are used to perform various actions on the frontend.

Event Handling: with v-on directive, it can listen to the events

Computed Properties: helps to listen to the changes made to the UI elements and performs necessary calculations

Components: custom elements that can be reused in HTML

Vue Components

Vue components are an abstraction that allows us to build large-scale applications composed of small, self-contained, and often reusable components.

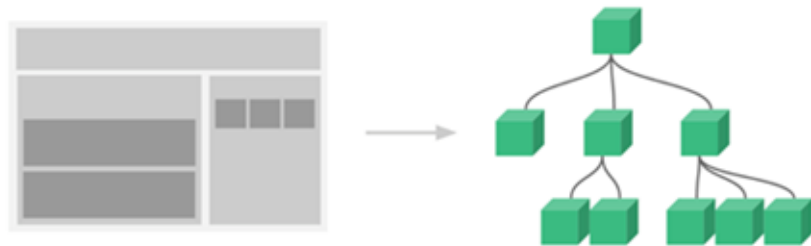
A component is a group of elements (i.e., template) together with the Vue instances with a name so that it can be reused.

In a large application, it is necessary to divide the whole app into components to make development manageable and scalable.

Vue components allow you to define your own reactive tags.

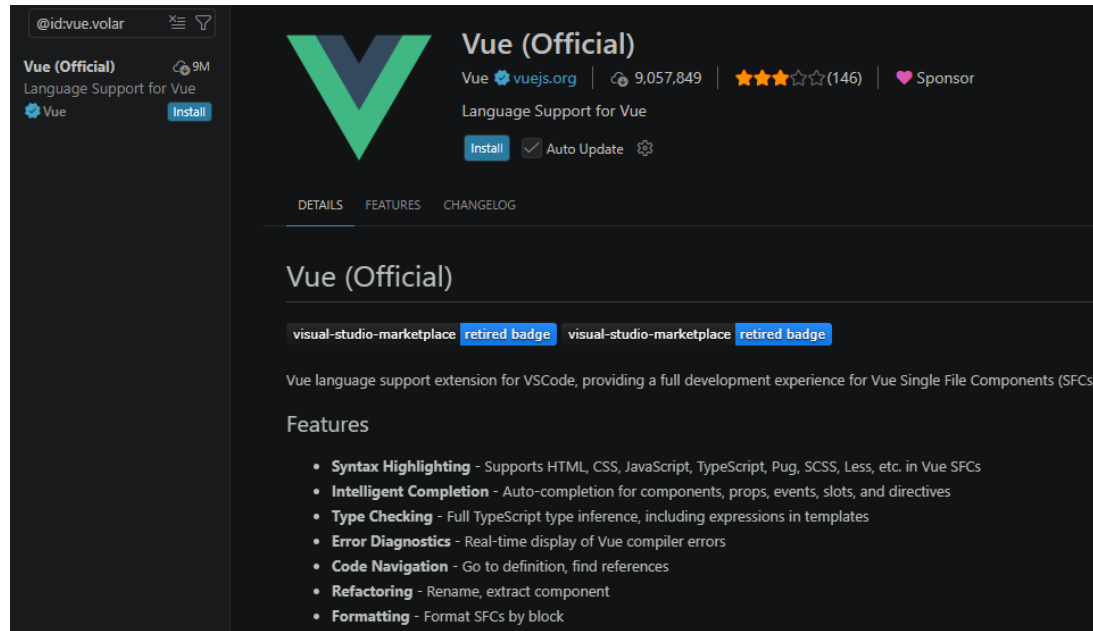
Example

```
<div id="app">  
  <app-nav></app-nav>  
  <app-view>  
    <app-sidebar></app-sidebar>  
    <app-content></app-content>  
  </app-view>  
</div>
```



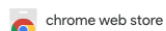
Vue – Official Extension for VS Code

Search for [Vue \(Official\)](#) or [vue.volar](#) in the VSCode extension and click install:



Vue.js devtools for Chrome

Install [Vue.js devtools](#) for chrome:



chrome web store



Search extensions and themes



Discover

Extensions

Themes



Vue.js devtools

[vuejs.org](#) 4.2★ (2.1K ratings) [Share](#)

Extension

Developer Tools

2,000,000 users

Add to Chrome

Install Node.js

Download and install Node.js:

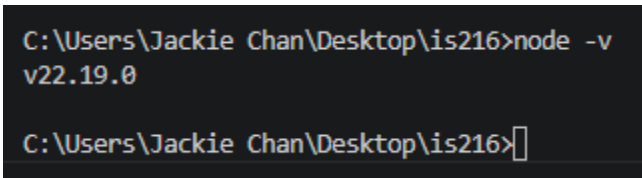
<https://nodejs.org/en>

After Node.js is installed, open the terminal and run the command:

`node -v`

If the version appears, it means node was successfully installed

Ensure that node version $\geq 22.12.0$



```
C:\Users\Jackie Chan\Desktop\is216>node -v  
v22.19.0
```

```
C:\Users\Jackie Chan\Desktop\is216>
```

Note: Node.js comes with NPM, which is a software package manager. NPM is needed to install pnpm, Vue CLI and other packages ...

Install pnpm

pnpm (Performant npm) is a fast, highly efficient JavaScript package manager designed as a drop-in replacement for npm and Yarn. It solves disk-bloat and slow installations by using a global, content-addressable storage to share packages across all projects, linking them via hard links and symlinks

Open your terminal (Command Prompt, PowerShell, or Mac terminal) and run:

```
npm install -g pnpm
```

Check if it was successfully installed:

```
pnpm -v
```

```
C:\Users\Jackie Chan\Desktop\is216>npm install -g pnpm  
  
added 1 package in 1s  
  
1 package is looking for funding  
  run `npm fund` for details  
  
C:\Users\Jackie Chan\Desktop\is216>
```

```
C:\Users\Jackie Chan\Desktop\is216>pnpm -v  
11.9.0  
  
C:\Users\Jackie Chan\Desktop\is216>
```

Quick Start (1)

Let's create a Vue app.

In the terminal, run

```
pnpm create vue@latest
```

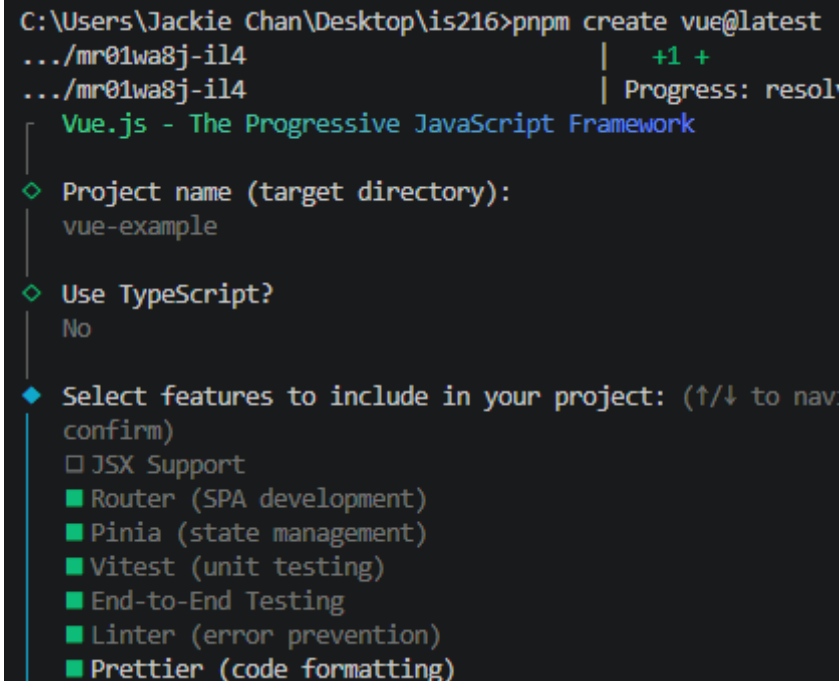
Project name -> vue-example

Use TypeScript? No

Select features -> press spacebar to select:

Router, Pinia, Vitest, End-to-End Testing,
Linter, Prettier

Press Enter

A terminal window showing the execution of the command 'pnpm create vue@latest'. The prompt is 'C:\Users\Jackie Chan\Desktop\is216>'. The command is entered and executed. The output shows the progress of creating a new Vue.js project. It asks for the project name (target directory), which is 'vue-example'. It then asks 'Use TypeScript?' with the answer 'No'. Finally, it asks to 'Select features to include in your project: (↑/↓ to navigate, space to select, enter to confirm)'. The features listed are: Router (SPA development), Pinia (state management), Vitest (unit testing), End-to-End Testing, Linter (error prevention), and Prettier (code formatting). All these features are selected with green checkboxes. A red bracket on the left side of the terminal output groups the feature selection step with the text 'Select features -> press spacebar to select:' from the main text on the left.

```
C:\Users\Jackie Chan\Desktop\is216>pnpm create vue@latest
.../mr01wa8j-il4 | +1 +
.../mr01wa8j-il4 | Progress: resol
Vue.js - The Progressive JavaScript Framework

◇ Project name (target directory):
  vue-example

◇ Use TypeScript?
  No

◆ Select features to include in your project: (↑/↓ to navigate, space to select, enter to confirm)
  ☐ JSX Support
  ☒ Router (SPA development)
  ☒ Pinia (state management)
  ☒ Vitest (unit testing)
  ☒ End-to-End Testing
  ☒ Linter (error prevention)
  ☒ Prettier (code formatting)
```

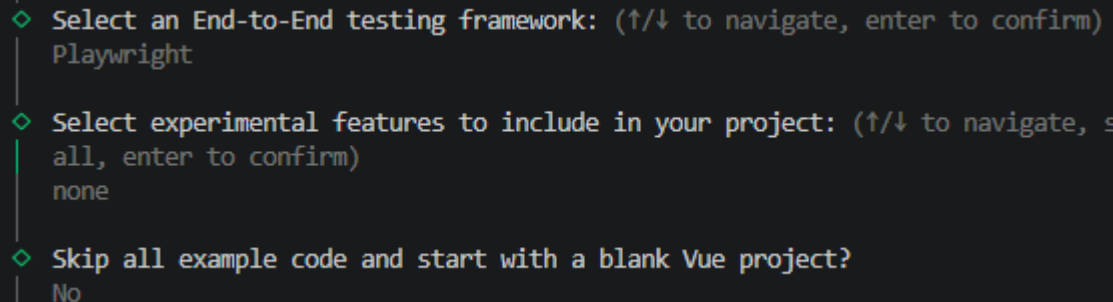
[follow this guide for detail](#)

Quick Start (2)

Select an End-to-End testing framework: Playwright

Select experimental features ... : none

Skip all example code ... ? No



```
◇ Select an End-to-End testing framework: (↑/↓ to navigate, enter to confirm)
  Playwright

◇ Select experimental features to include in your project: (↑/↓ to navigate, s
  all, enter to confirm)
  none

◇ Skip all example code and start with a blank Vue project?
  No
```

Quick Start (3)

Vue project should be created in your workspace.

Run the following commands (one by one):

```
cd vue-example
pnpm install
pnpm format
pnpm dev
```

After running the above commands, vue app is deployed. Click on this link to view the app in the browser

Note: the link with “/__devtools__/” will open the Vue dev console (for debugging purposes).

```
Scaffolding project in C:\Users\Jackie Chan\Desktop\is216\vue-example...
|
| Done. Now run:
|
| cd vue-example
| pnpm install
| pnpm format
| pnpm dev
```

```
VITE v8.1.0 ready in 2457 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ Vue DevTools: Open http://localhost:5173/__devtools__/ as a separate window
→ Vue DevTools: Press Alt(⌘)+Shift(⇧)+D in App to toggle the Vue DevTools
→ press h + enter to show help
```

To stop the server, press **Ctrl + c**

To start again, run the command **pnpm dev**

Structure of a Vue App (1)

node_modules/: stores all external libraries and dependencies your project needs to function. This is auto-generated by your package manager and shouldn't be edited directly

public/: stores static assets like favicon.ico

src/: The heart of your application. This contains:

assets/: files that need to be processed by your bundler, such as raw CSS/SCSS files, fonts, and images.

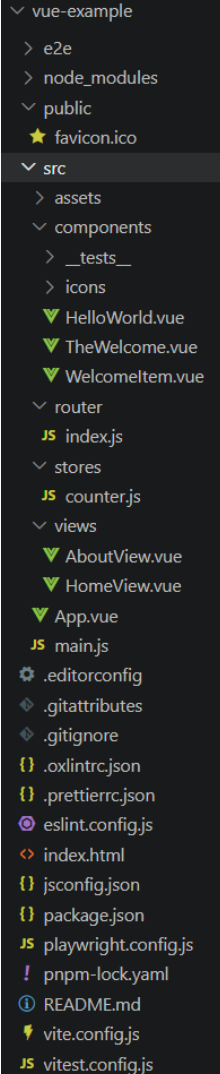
components/: contains modular, reusable pieces of your UI components (e.g., headers, footers, navigation bars, and cards).

router/: contains routing configuration (index.js) that maps URL paths to specific views.

stores/: holds state management logic (using **Pinia**) for centralized data sharing across your SPA.

views/ (or pages/): Contains components that represent full pages or major UI sections, typically mapped to routing paths (e.g., Home.vue, About.vue).

Additionally you may create **utils/ (or helpers/)** folder to store miscellaneous helper functions, such as date formatters or math calculators, used throughout the app



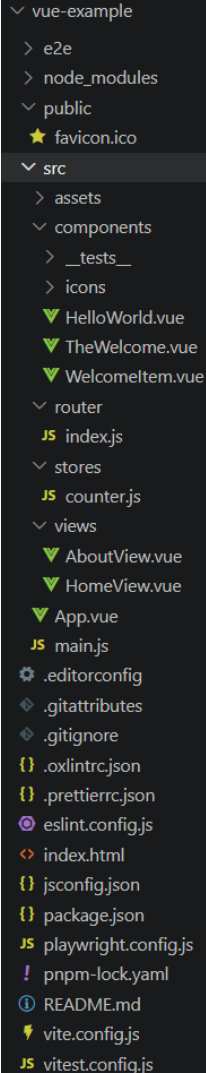
Structure of a Vue App (2)

index.html: the landing page of your web app. It is also the only .html page (Single Page App). Notice that it

- contains `<div>` element with id “app”. This is where all the contents (Vue components) will be loaded
- imports “main.js” file

main.js: imports libraries and Vue components; initialize Vue app instance (App.vue) and attaches it to the DOM (`<div id='app'></div>`) in index.html

App.vue: the master/root component. All other components are typically nested inside this single-file component.



Structure of a Vue App (3)

Following are configuration files that we don't usually need to bother with:

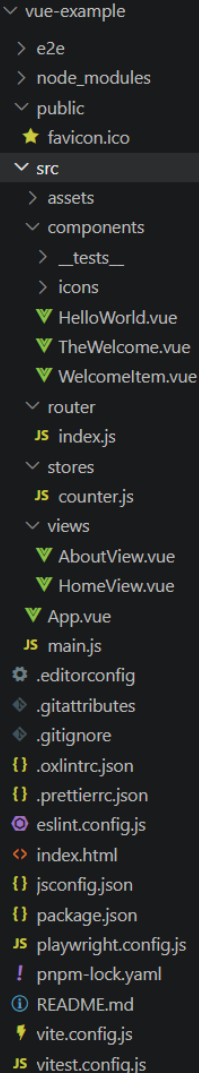
package.json: lists your project dependencies, scripts, and version metadata

vite.config.js: bundler configuration file that lets you define plugins, proxies, and build optimizations.

vitest.config.ts: configuration file for Vitest, a fast, Vite-native testing framework for Vue. It defines how your **unit and component tests** are configured, executed, and integrated into your build pipeline. Note: *.ts denotes a typescript file, which is a typed, compiled language built on top of JavaScript.

playwright.config.js: configuration file for Playwright, a powerful end-to-end (E2E) testing framework. Unlike unit tests that check small pieces of code, E2E tests launch real browsers to simulate actual user journeys—like logging in, clicking buttons, and completing checkouts. This file controls exactly how those browsers behave and how your tests execute.

vue.spec.js: boilerplate, starter test file auto-generated when you scaffold a Vue app using tools like Vite with End-to-End (E2E) testing selected



Routing (1)

Routing on **the server side** means the server is sending a response based on the URL path that the user is visiting. When we click on a link in a server-rendered web app (e.g., Express.js + MVC + ejs template engine), the browser receives an HTML response from the server and reloads the entire page with the new HTML.

Routing on **the client side** means the client side app (e.g., Vue's Single-Page Application) can intercept the navigation, dynamically fetch new data, and update the current page (typically with a new Vue component), without full page reloads.

In Vue's Single-Page Application (SPA), this is done by using **Vue Router**.

Routing (2)

Vue Router is a Vue Library that provides two components: RouterLink and RouterView.

RouterLink is used to create links (Instead of using regular <a> tags)

RouterView component tells Vue Router (the location) where to render the current route component.

Syntax

<RouterView />

```
<template>  
  <RouterView />  
</template>
```

<RouterLink to=<path> > Label </RouterLink>

```
<template>  
  <RouterLink to="/">Go to Home</RouterLink>  
  <RouterLink to="/about">Go to About</RouterLink>  
</template>
```

Routing (3)

Check out **App.vue** and **router/index.js** in the example app that we created based on the Quick Start slides

App.vue provides a default template which use `<RouterView />`

⇒ This will link to `router/index.js`

In `App.vue` or any other component nested in `App.vue`, use `<RouterLink></RouterLink>` to map the URL to the specific Vue components. For example, “`http://localhost:5173/about`” maps to “`AboutView.vue`” component, which will be rendered in the `<RouterView />` location in `App.vue`

```
<nav>
  <RouterLink to="/">Home</RouterLink>
  <RouterLink to="/about">About</RouterLink>
</nav>
</div>
</header>

<RouterView />
```

App.vue

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomeView,
    },
    {
      path: '/about',
      name: 'about',
      // route level code-splitting
      // this generates a separate chunk (About.[hash].js)
      // which is lazy-loaded when the route is visited.
      component: () => import('../views/AboutView.vue'),
    },
  ],
})

export default router
```

router/index.js

Hello World

Check out Example1.vue component in given **example** folder

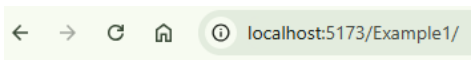
Example: *Example1.vue*

In the terminal, go to the example folder and run

```
pnpm dev
```

Click on the localhost link

In the browser, click on “Example1” and observe the output



Hello World

DevTools

Open the devtools by clicking the link with
[/_devtools_/](#)

Example: *Example1.vue*

Click on the buttons in the left panel and explore



Component: Syntax

Example: *Example2.vue*

Components contain 3 things. Yes you guessed them right. They are JavaScript, HTML, and CSS 😊

```
<script setup> ** Vue/JS stuff ** </script>
```

```
<template>** HTML stuff ** </template>
```

```
<style scoped>** CSS stuff ** </style>
```

Note: “scoped” means the styling is applicable only to this particular component.

Component: Syntax

ref: Vue's library function used to create reactive variables. It takes the argument and returns it wrapped within a ref object with a **.value** property

Other Vue libraries that are typically imported include:

`onMounted`, `computed`, `defineProps`, `defineEmits`, ...

Note: for objects, `reactive`, instead of `ref`, can also be used. But we shall cover this function in this course.

For more info: <https://vuejs.org/guide/essentials/reactivity-fundamentals.html#reactive>

Example: *Example2.vue*

```
// import libraries
```

```
import { ref } from 'vue'
```

```
// define data
```

```
const name = ref("John")
```

```
const phone = ref(123456)
```

```
// use data
```

```
console.log(name.value)
```

```
// define method(s)
```

```
function getPhone() {
```

```
  // method doesn't necessarily 'return'
```

```
  return "This is my number " + phone.value
```

```
}
```

Data Binding (1)

The most basic form of data binding is text interpolation using the “Mustache” syntax (double curly braces):

`{{ ... }}` tells Vue to take everything inside them and parse it as code.

Name : John

Phone : This is my number 123456

Example: *Example2.vue*

```
<h1> Name : {{name}} </h1>
```

```
<h1> Phone : {{getPhone()}} </h1>
```

In the above example, *name* will be replaced with the value of the name property of the vue instance.

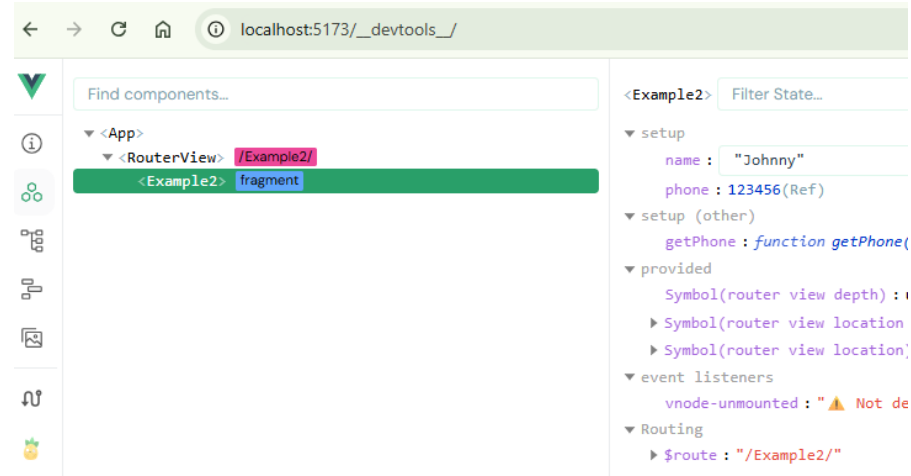
getPhone() will be replaced with the outcome of the invocation of vue instance’s *getPhone* method.

Data Binding (2)

The value inside `{{ ... }}` will be updated whenever there is a change in property.

Experiment:

1. In devtools, select *Example2* component
2. Change name property to "Johnny"
3. Observe that the page is updated automatically



Vue Debugging 101

Use Chrome console and devtools to debug

Enter variable names and check if you get expected results

If you run into something you can't figure out, or you find a particularly nasty error, check out <https://forum.vuejs.org/c/help>

Two common issues

Symptom: "Uncaught SyntaxError: Unexpected identifier"

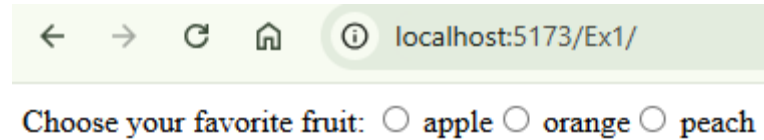
Illness: usually typo error, missing comma or braces, or hidden strange characters

Symptom: "[Vue warn]: Property or method \"propertyname\" is not defined"

Illness: something was not defined in the options object (property or method)

Exercise 1

In Ex1.vue, the HTML part of the page is given. Program the script part by so that the page looks like the following.



Note: you need to install/include the node_modules first. In the terminal,
cd resource
pnpm dev

Vue Directives

v-bind	Bind an attribute value with a vue variable	<code></code>
v-html	Print a given vue variable as HTML	<code><div v-html="message"></div></code>
v-on	Listen to DOM events and run code	<code><button v-on:click="msg = 'hello'">Add</button></code>
v-for	Loop through items in a Vue parameter – list (items)	<code><li v-for="item in items"> </code>
v-if v-else	Remove/insert an element based on the truth value of Vue parameter	<code><div v-if="seen">You either see this</div> <div v-else>or this!</div></code>
v-model	Bind Vue parameters to the values of input elements such as text, radio, and checkbox	<code><input v-model="msg"> <select v-model="option"></code>

Directives are special attributes with the v-prefix, used to reactively apply side effects to the DOM when the value of its expression changes.

Conditional Rendering (1)

The directive `v-if`, `v-else` is used to conditionally render a block. The block will only be rendered if the directive's expression returns a truthy value.

A `v-else` element must immediately follow a `v-if` or a `v-else-if` element - otherwise it will not be recognized.

*explore `v-else-if` on your own

Example: *Example3.vue*

```
<p v-if="x==1">Now you see me</p>
```

```
<p v-else>You don't see me</p>
```

Here, the `v-if` directive would remove/insert the different `<p>` element based on the truthiness of the expression `x==1`.

Note that `x` should be a reactive variable.

Directive attribute values are expected to be a single JavaScript expression.

Conditional Rendering (2)

To toggle more than one element, we can use v-if on a `<template>` element, which serves as an invisible wrapper.

The final rendered result will not include the `<template>` element.

Example: *Example4.vue*

```
<template v-if="ok">
```

```
  <h1>Title</h1>
```

```
  <p>Paragraph 1</p>
```

```
  <p>Paragraph 2</p>
```

```
</template>
```

Attribute Binding (1)

The value of HTML attributes, e.g., id, value, src, href, ..., can be bound to the Vue component's property

Note: Mustaches {{ ... }} cannot be used for binding inside HTML attributes. Use v-bind directive, instead.

Example: *Example5.vue*

```

```

This directive instructs Vue to keep the image element's width attribute in sync with the component's property x.

Shorthand

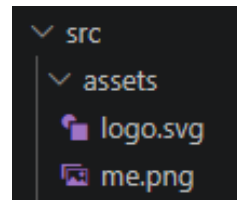
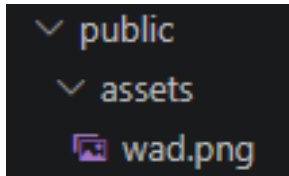
```

```

Importing an image

You can place static assets like images or text files in `/public` folder or `/src/assets` folder

Import and use these assets: *Example5.vue*



```
<script setup>
  import { ref } from 'vue'

  import wad from '/assets/wad.png' // public/assets/ folder
  const me = ref('/src/assets/me.png') // src/assets/ folder

</script>

<template>
  <hr>
  <hr>
</template>
```

Static assets in src/assets vs public folders

Both `src/assets` and `public` folder can store static assets like images and text files but there are some differences:

	<code>src/assets</code>	<code>public</code>
referencing	Referenced via relative paths, e.g., <code></code>	Referenced relative to the web root, e.g., <code></code>
Build process	Compiled, optimized, and minified by the bundler	Copied as-is to the output directory, e.g., <code>dist</code>
Dead code elimination	Unused assets are excluded from the final build	All files are included in the build

Attribute Binding (2)

You can also bind **multiple attributes** using an object

Example

```
const obj = {
```

```
  src: 'x',
```

```
  width: 'y',
```

```
}
```

```
===
```

```
<img v-bind="obj" >
```

Exercise 2

In Ex2.vue, the script part is given. Complete the HTML part such that the image is displayed with data defined in the script part.



Use the given 'width' and 'height' properties to set the image width and height. Try using both single attribute binding and multiple attribute binding

Attribute Binding (3)

Boolean attributes are attributes that can indicate true / false values by their presence on an element.

For example, disabled is a common Boolean attribute:

Example: *Example6.vue*

```
const isButtonDisabled = ref(true)
```

===

```
<button :disabled="isButtonDisabled">Button</button>
```

Form Input Binding

Use v-model to create **two-way data bindings** on form input, textarea, and select elements.

Fyi (non-examinable) : v-model may use different **properties** and emits different **events** for different input elements.

- text and textarea elements use **value** property and **input** event;
- checkboxes and radio buttons use **value** (or **checked** property if value is not defined) and **change** event;
- select fields use **value** as a prop and **change** as an event.

Example: *Example7.vue*

```
const username = ref("")
```

```
===
```

```
Name: <input type='text' v-model='username' >
```

```
Your name is {{ username }}
```

Exercise 3

In Ex3.vue, a dropdown list is provided to allow users to login with either username or email. Render the page different depending on the selection, i.e., show an input for an email or a username accordingly.

Hint: use v-model, v-if, and v-else to accomplish the task.

Please select your login option

Username login ▼

Username

Please select your login option

Email login ▼

Email

Class and Style Binding (1)

A common need for data binding is manipulating an element's class and its inline styles, to control CSS

Use **v-bind** to bind **class** and **style** attributes

Syntax:

```
v-bind:class="{className' : isActive}"
```

```
v-bind:class = "[prop1, prop2]"
```

```
v-bind:style="{css-property' : prop}"
```

Example: *Example8.vue* and *Example9.vue*

Notice the CSS code written in `<style scoped>` in *Example8.vue* (internal styling) and the absence of it in *Example9.vue*

Class and Style Binding (2)

Binding to Objects

We can pass an object to `:class` to dynamically toggle classes:

```
<div v-bind:class="{ 'className' : isActive }"></div>
```

You can have multiple classes:

```
<div  
  class="static"  
  :class="{ 'demo': isActive, 'text-danger': hasError }"  
></div>
```

Example: *Example8.vue*

Don't forget the quotes.

isActive and hasError are
the Vue component's
properties

Class and Style Binding (3)

Binding to Arrays

We can pass an array to `:class` to apply a list of classes:

Example: *Example8.vue*

```
const activeClass = ref('active')  
const errorClass = ref('text-danger')
```

===

```
<div :class="[activeClass, errorClass]"></div>
```

Class and Style Binding (4)

Binding Inline Styles

Similar to class attribute, we can also bind to style attribute with an object

Example: *Example9.vue*

```
const activeColor = ref('red')
const fontSize = ref(30)

===

<div :style="{
  color: activeColor,

  fontSize: fontSize + 'px'
}">

</div>
```

Exercise 4

Part 1:

Add code in Ex4.vue such that when you click “Change Color”, it changes to red from blue or to blue for red alternately.



Part 2:

Add code in Ex4.vue such that when you click “Change Text Color”, it changes to red from blue or to blue for red alternately.



Use v-bind with class and style bindings

Challenge: you will notice that **your buttons** does not have colors. Debug!!!

Routing: with Parameter

In the routes, we can pass a parameter.

For example,

```
{path: "/products/:id", component:  
  "ProductsView" }
```

When the user visits the path, e.g.,
<http://localhost:5173/products/1>, the router
launches `ProductsView` component, passing it the
parameter `id` with the value of `1`

That is, in `ProductsView` component, the parameter
can be accessed via `route.params.xx`

Example *Example10.vue*

```
import { useRoute } from 'vue-router'
```

```
// 1. Initialize the route hook  
const route = useRoute()
```

```
// 2. Read the parameter (Note: route  
params are always strings)  
const productId = route.params.id
```

Routing: Redirect (1)

In routing, we can redirect a given URL to another URL.

Example router/index.js

For example,

```
{ path: "/invalid", redirect: "/Example1" }
```

When the user visits an invalid URL `/invalid` (e.g., <http://localhost:5173/invalid>), the router automatically redirects to `/Example1`

Use case:

- Redirect to a default landing page when the user visits the site
- Redirect to a login page after user registration

Routing: Redirect (2)

Redirect can also be implemented programmatically using `router.push(/new_url)`

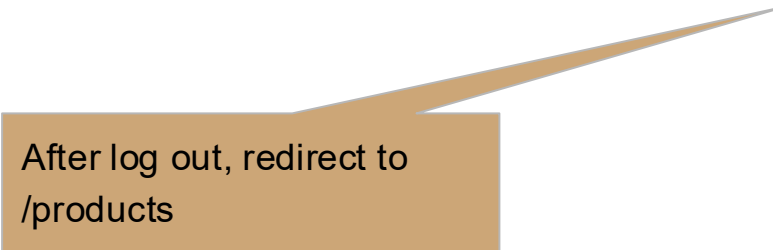
When `router.push` executes, Vue Router changes the browser's current address path to `/new_url` and loads the component mapped to that route.

Example *Example11.vue*

```
import { useRouter } from "vue-router"
const router = useRouter()

function logout() {
  ... // logout logic

  router.push("/Example1");
}
```



After log out, redirect to
`/products`

Take Away Message

With vanilla Javascript, you will constantly go from HTML to Script and back (via DOM, search from the root of the DOM tree, using *document.getElement...*).

With Vue.js, you declare the connection between HTML and Script and focus more on the logic in Vue.

Put it simply: no more *document.getElement...*